K-axis Grouped GEMM

MoE weight gradients with experts stacked along the reduction axis

DeepSeek-AI · DeepGEMM (MoE weight backward)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

GEMM & MoE Kernels · github.com/deepseek-ai/DeepGEMM

Each expert trains with three FP8 GEMMs

- DeepSeek-V3: all three in FP8
- Fprop, Dgrad: ragged M rows (deck 33)
- Wgrad: ragged K, the summed axis

GEMM	per expert e	ragged axis	DeepGEMM
Fprop	$Y = X\ W^T$	 M rows	`m_grouped nt`
Dgrad	$dX = dY\ W$	 M rows	`m_grouped nn`
Wgrad	$dW = dY^T\ X$	 **K (summed)**	**`k_grouped`**

Wgrad is a third of expert GEMM work

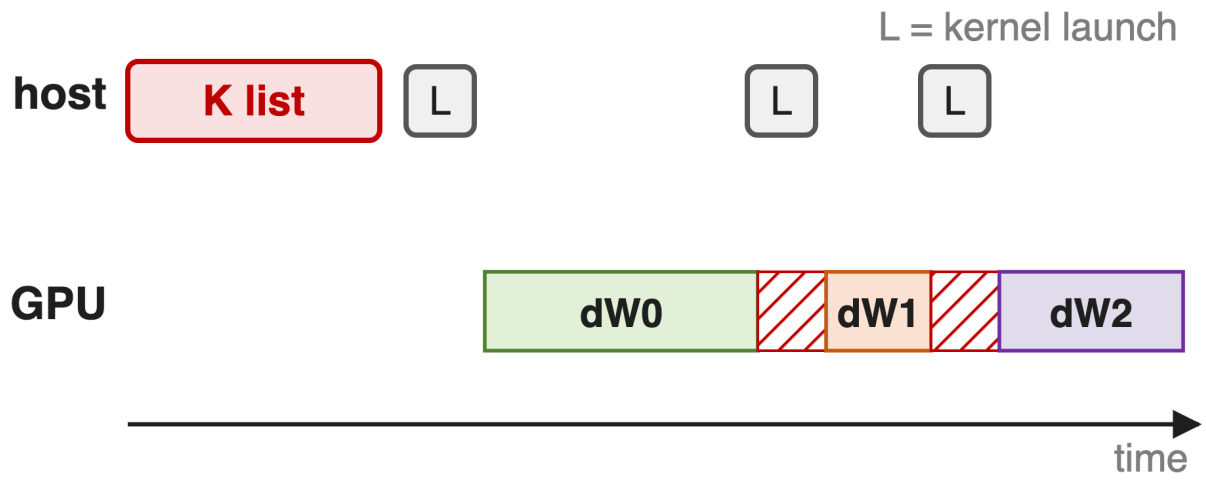
1 / 3

of each expert's training GEMM FLOPs is the weight gradient, the one GEMM that sums over the ragged token count

Analytic: Fprop, Dgrad and Wgrad each cost 2·M·N·K FLOPs. DeepSeek-V3 runs all three in FP8 (arXiv 2412.19437 §3.3.1)

Ragged K defeats per-expert loops and padding

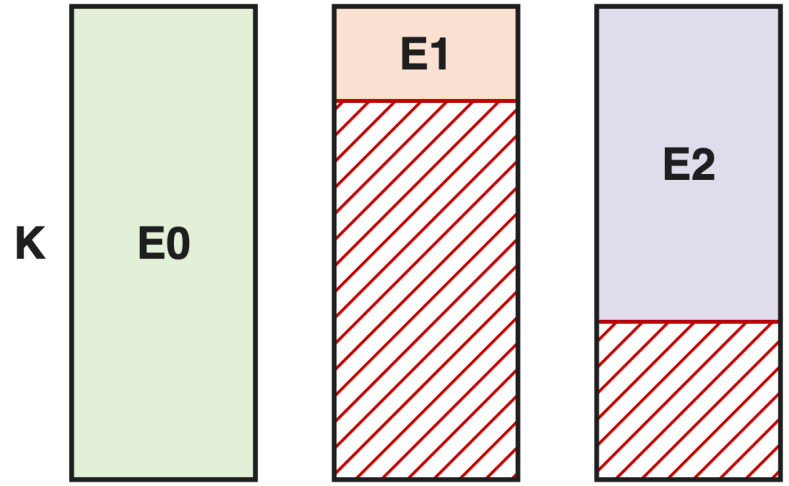
(a) One Wgrad GEMM per expert



G launches per weight per layer

token counts must reach the host

(b) Pad every expert to the longest



zero tokens cost full FLOPs

K = longest expert for all

The ragged axis is the one Wgrad sums over, so M-grouping cannot express it

Insight: in Wgrad, experts differ only in K

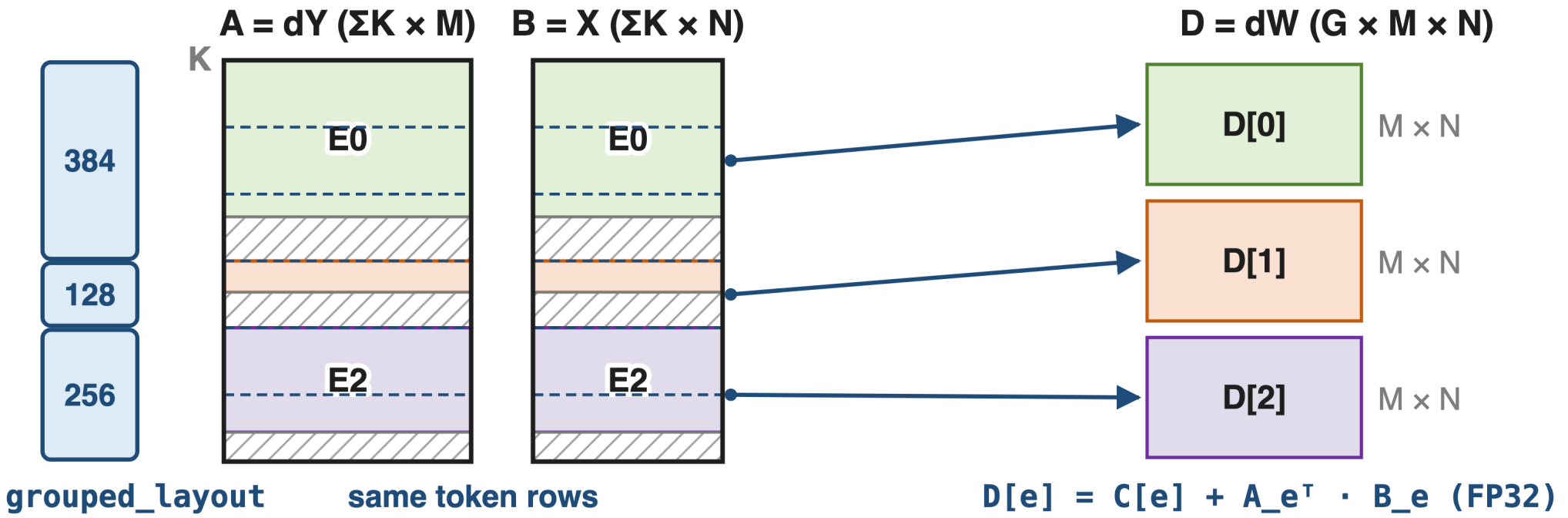
- Every expert: same output dW_e, M × N
- Only its token count K_e varies

Insight: in Wgrad, experts differ only in K

- Every expert: same output dW_e, M × N
- Only its token count K_e varies

Stack tokens along K, align each expert to the 128-token block, let each tile sum only its expert

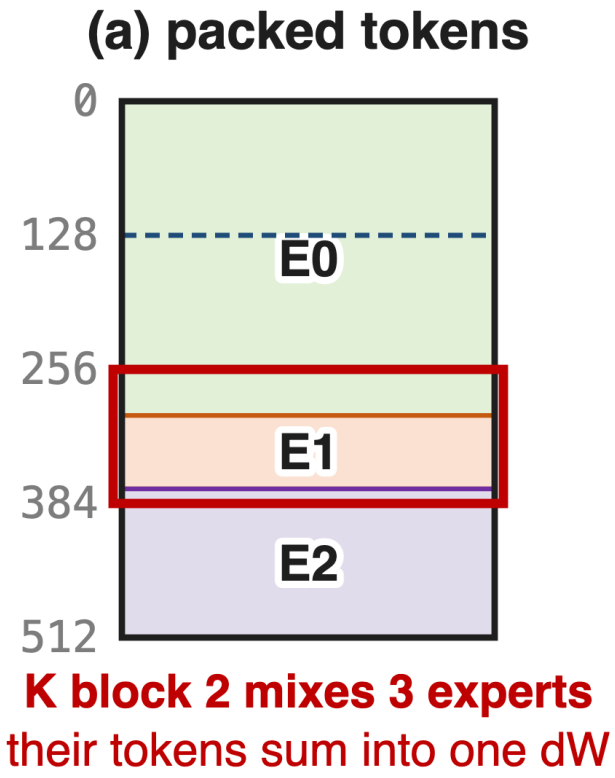
Stack experts along K, one output per expert



k_grouped_fp8_gemm_tn_contiguous((A, sfa), (B, sfb), D, ks_cpu, grouped_layout, c=C) · A FP8 [ΣK, M] · B FP8 [ΣK, N] · D FP32 [G, M, N] · SM100

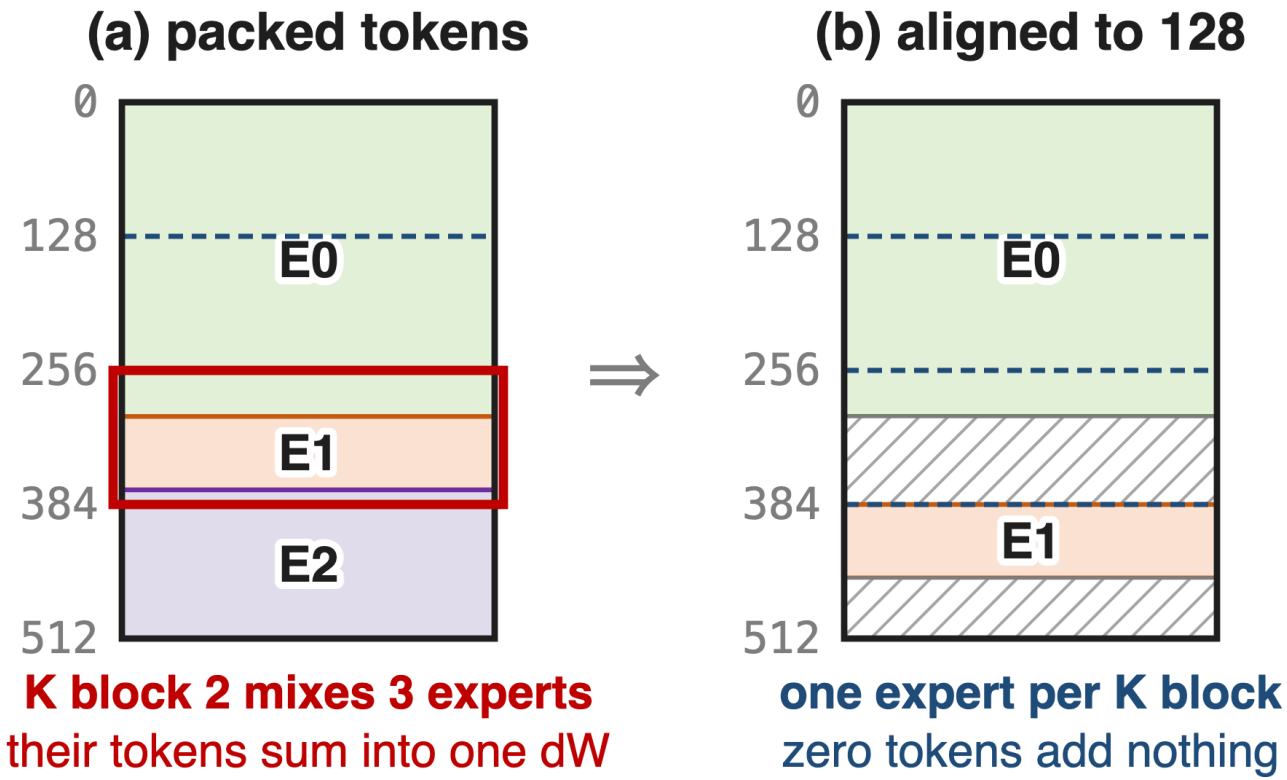
❶ Align each expert to the 128-token K block

- A K block feeds one dot product
- Packed tokens mix experts' gradients

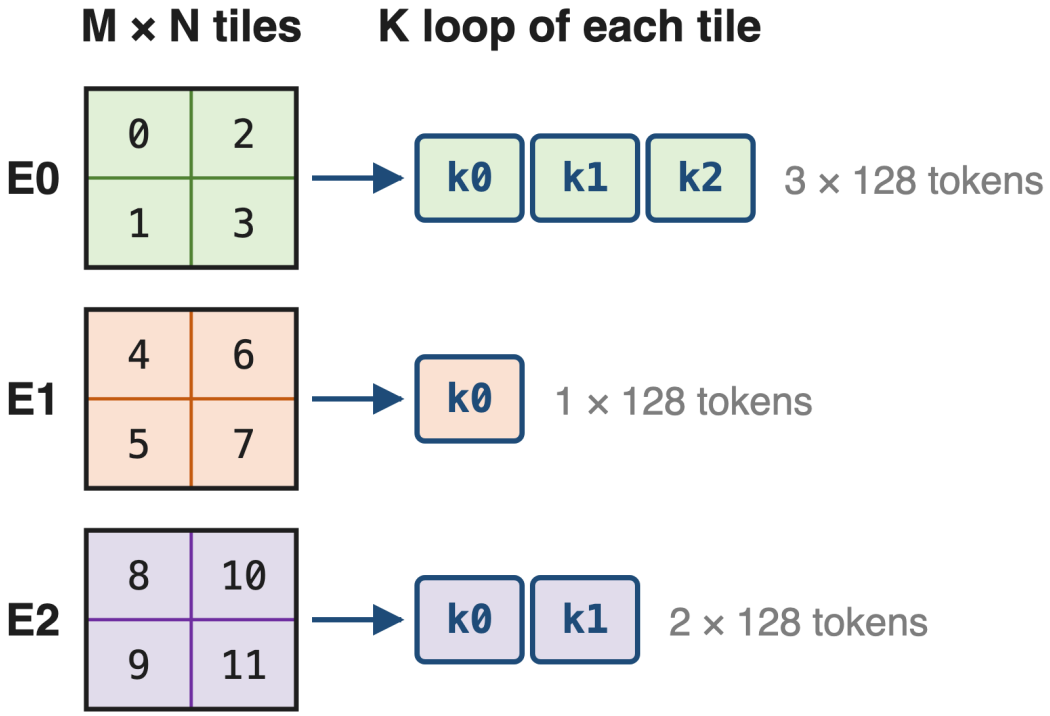


❶ Align each expert to the 128-token K block

- A K block feeds one dot product
- Packed tokens mix experts' gradients
- Pad each expert to **128** tokens with zeros
- Same alignment as the forward (deck 33)



❷ Every expert gets the same tile grid



one CTA per SM (persistent)

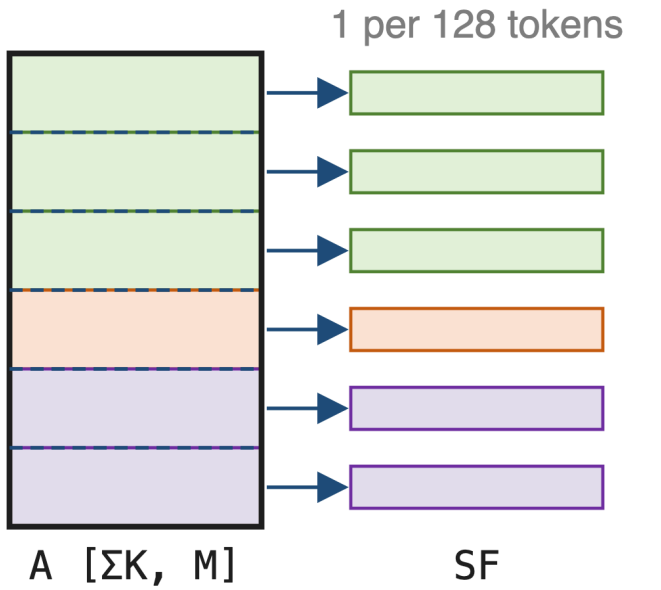
```
for t in range(sm, G*tiles, SMs):        all experts
  g, m, n = split(t)                     same grid
  acc = C[g][m, n]                       fused accum
  for kb in k_blocks(g):                 own tokens
    acc += A[kb,m].T @ B[kb,n]           FP8 MMA
  D[g][m, n] = acc                       FP32 store
```

k_blocks(g) from grouped_layout · start aligned to 128

One launch for all experts, and C + $A^T B$ folds gradient accumulation into the FP32 output

❸ Scales and offsets follow the token axis

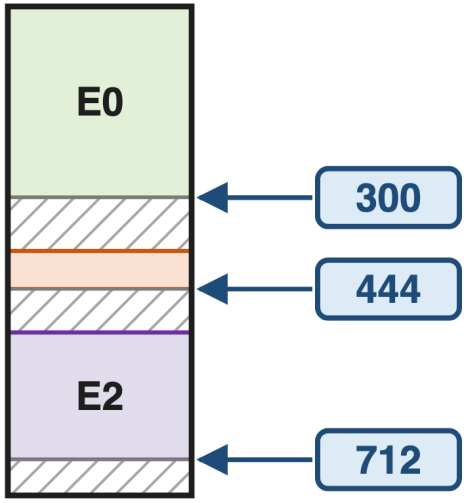
(a) FP8 scales



packed per expert (UE8M0 on SM100)

(b) psum offsets on GPU

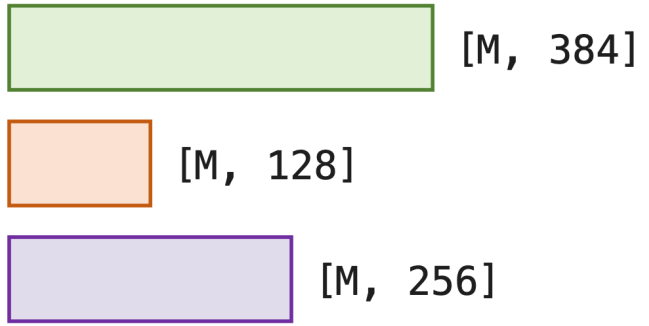
ends



next start = align(end, 128)

no host K list (SM100)

(c) SM90: K-major copies

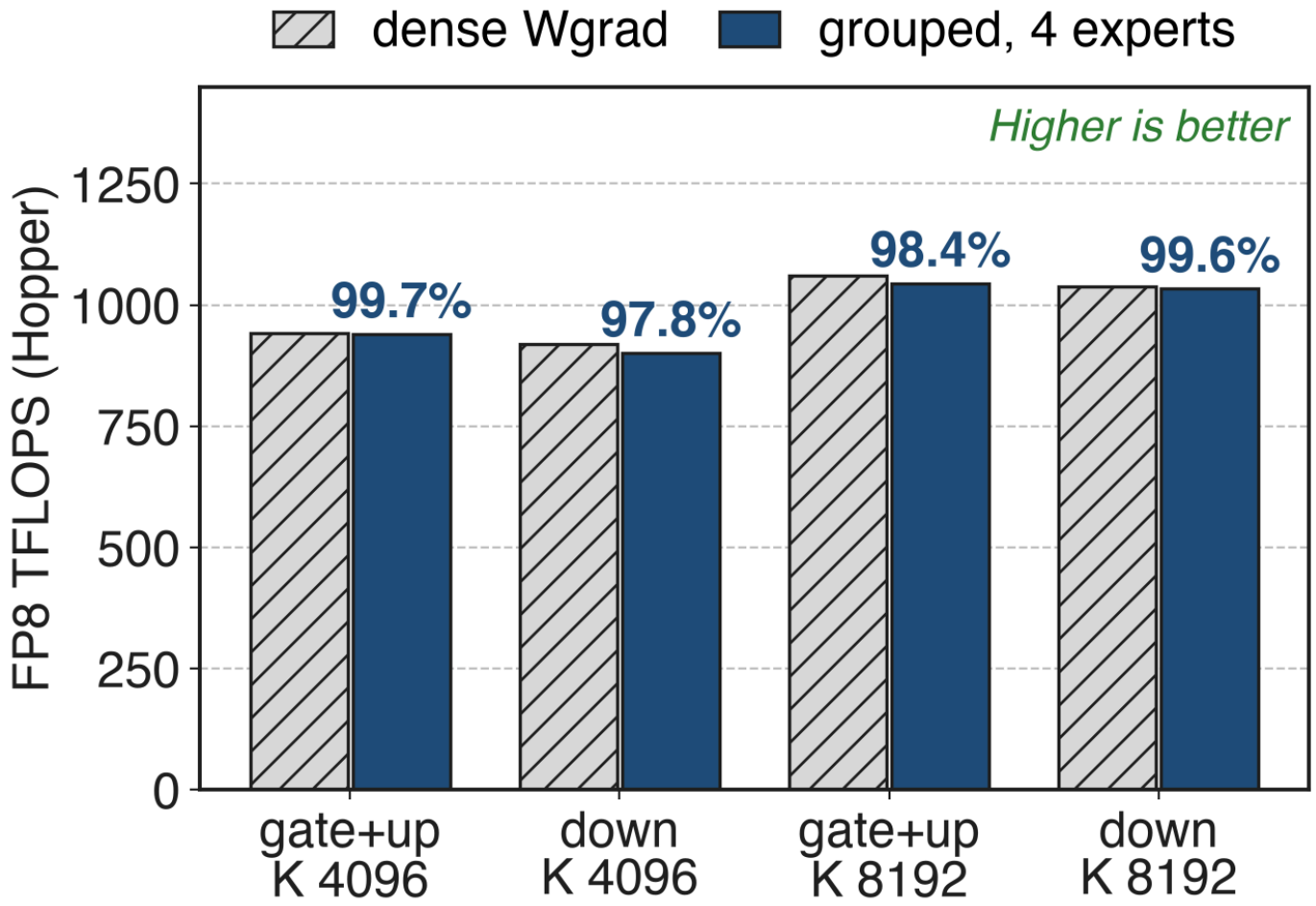


TMA descriptor in SMEM
addr + stride per expert

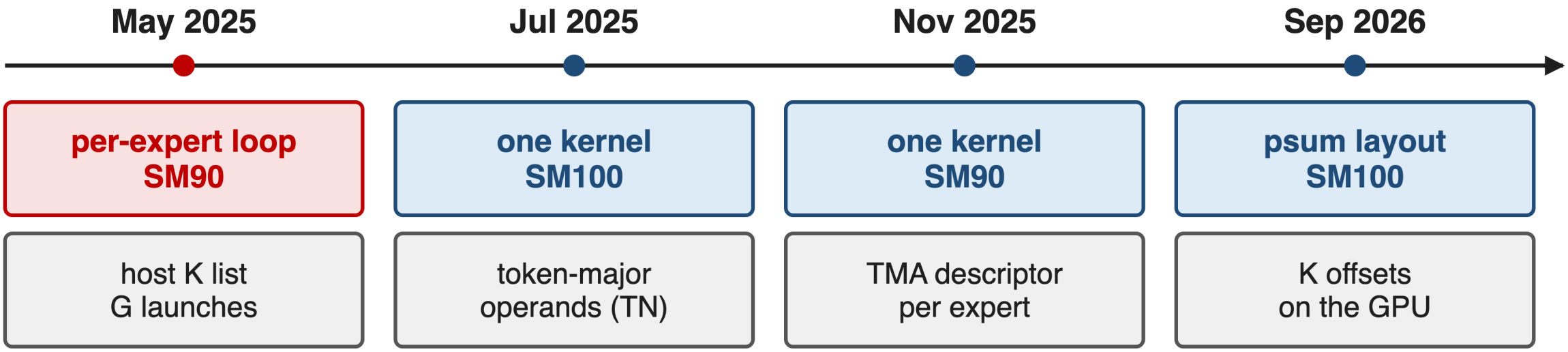
Recipe (1, 1, 128): one scale per channel per 128 tokens on both operands

Grouped Wgrad keeps ≥ 97.8% of dense speed

- **900–1044** TFLOPS on V3 expert shapes
- **~20%** over internal CUTLASS
- May 2025 loop, kernel time only



From a host loop to one GPU-driven kernel



PRs #95 · #112 · #231 · #432

Sept 2026 release: MoE backward GEMMs about 20% faster

The cost: token-axis casts and shared shapes

- **Token-axis FP8 casts**: X and dY per 128 tokens
- Shared M, N; ≤ **127** pad tokens per expert
- SM90: K-major copies, host K list, FP32 C
- No public numbers for the one-kernel versions

Megatron-Core also pads tokens per expert to 128 for FP8 Wgrad and CUDA Graphs

Summary

- Wgrad: **1/3** of expert GEMM FLOPs, summed over ragged tokens
- **Insight**: experts differ only in K: stack tokens, align to **128**
- One kernel: same tile grid per expert, FP32 accumulate
- **900–1044** TFLOPS, **≥ 97.8%** of dense; **~20%** faster in 2026

https://github.com/deepseek-ai/DeepGEMM